Programación I

Contadores y Acumuladores

Aprenderemos a gestionar variables que nos permiten llevar la cuenta de sucesos o sumar valores de manera progresiva. Estas herramientas son esenciales cuando necesitamos procesar múltiples datos para obtener resultados como totales o promedios.

Introducción

En programación, muchas veces no basta con procesar un dato de forma aislada; necesitamos "recordar" cuántas veces sucedió algo o cuánto suma un conjunto de valores. Para esto utilizamos variables con roles específicos: el **contador** y el **acumulador**.

Ambos conceptos se basan en la autorreferencia: la variable aparece a ambos lados del signo de asignación (ej. A = A + 1), lo que permite que el nuevo valor dependa del valor anterior.

Contadores

Un **contador** es una variable cuyo valor se incrementa o decrementa en una **cantidad constante** (generalmente 1) cada vez que ocurre un evento.

- **Inicialización**: Es crítico asignarles un valor inicial (normalmente 0) antes de empezar el conteo.

- **Uso común**: Contar alumnos presentes, cantidad de ventas, o iteraciones en un bucle.

Implementación

Herramienta	Sintaxis de Incremento	Ejemplo
Diagramación	**cantidad = cantidad + 1**	**alumnos = alumnos + 1**
C++	**cant++** ó **cant += 1**	**conteo++**

Operaciones de un contador

OPERADOR	SIGNIFICADO
++	Aumenta uno a una variable
--	Disminuye uno en una variable

OPERACIÓN	EQUIVALENCIA
A ++	A = A + 1
A --	A = A – 1

Acumuladores

Un **acumulador** es una variable que aumenta o disminuye en **una cantidad variable**. Se utiliza para obtener totales de una serie de valores.

- **Diferencia clave**: Mientras el contador siempre suma "1", el acumulador suma el valor de otra variable (ej. un precio, una nota, un peso).

- **Inicialización**: Al igual que el contador, debe arrancar en 0 para no sumar valores basura.

Implementación

Herramienta	Sintaxis de Incremento	Ejemplo
Diagramación	acum = acum + valor	totalSueldos = totalSueldos + sueldo
C++	acum += valor	totalSueldos += sueldo

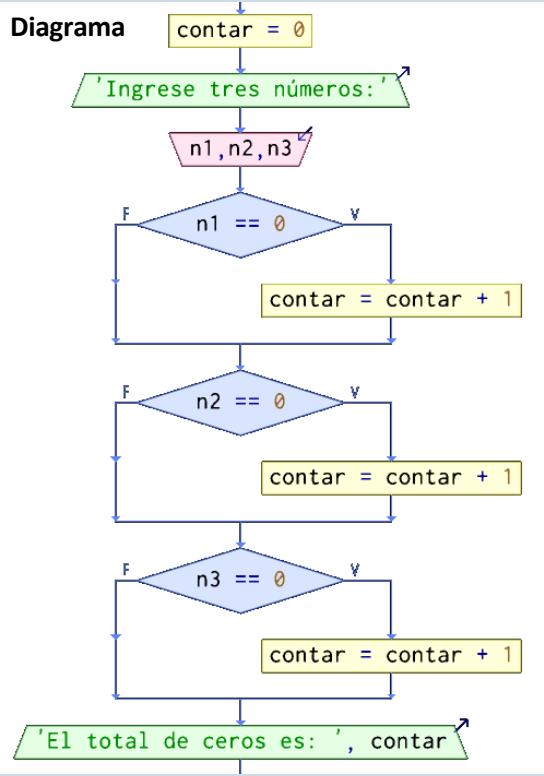
Operaciones de un Acumulador

OPERADOR	SIGNIFICADO
+=	Aumenta una cantidad a una variable
-=	Disminuye una cantidad a una variable
*=	Multiplica por una cantidad a una variable
/=	Divide por una cantidad a una variable

OPERACIÓN	EQUIVALENCIA
A += B	A = A + B
A -= B	A = A – B
A *= B	A = A * B
A /= B	A = A / B

Ejemplo Práctico: Cuantos números son cero

Problema: Ingresar 3 números y mostrar cuantos son ceros.

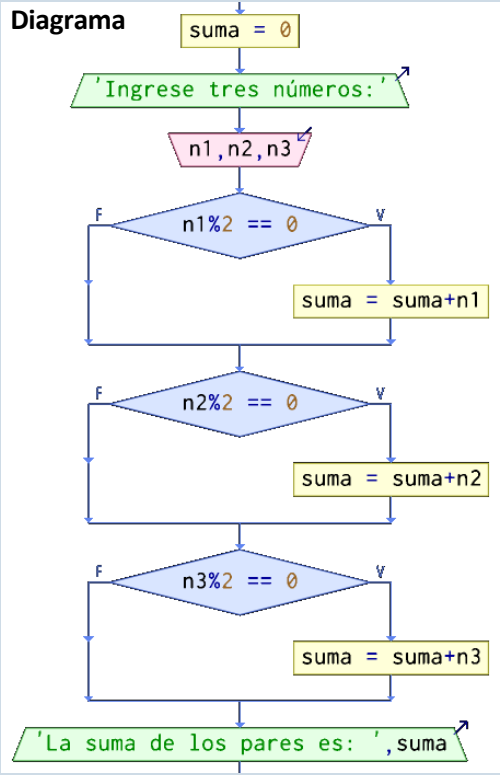



```
int n1, n2 n3;
int contar = 0; // El contador se inicializa en 0
cout << "Ingrese tres numeros: ";
cin >> n1;
cin >> n2;
cin >> n3;
if (n1 == 0) {
    contar ++; // Incremento en 1
}
if (n2 == 0) {
    contar ++;
}
if (n3 == 0) {
    contar ++;
}
cout << “El total de ceros es: " << suma;
```

Código C++

Ejemplo Práctico: Suma de Números Pares

Problema: Ingresar 3 números y mostrar la suma de aquellos que sean pares.




```cpp
int n1, n2 n3;
int suma = 0; // El acumulador se inicializa en 0
cout << "Ingrese tres numeros: ";
cin >> n1;
cin >> n2;
cin >> n3;
if (n1 % 2 == 0) {
    suma += n1; // Uso de asignación compuesta
}
if (n2 % 2 == 0) {
    suma += n2;
}
if (n3 % 2 == 0) {
    suma += n3;
}
cout << "La suma de los pares es: " << suma;
```

Código C++

Errores comunes

- ✓ **Olvidar la Inicialización**: Si no igualas el contador o acumulador a 0 al principio, el programa sumará sobre un valor desconocido ("basura" en memoria), dando resultados incorrectos.

- ✓ **Confundir el Rol**: Usar un contador (c = c + 1) cuando se requiere sumar montos variables (c = c + precio).

- ✓ **Ubicación Incorrecta**: Colocar la inicialización dentro de un bucle, lo que provoca que la variable se reinicie a cero en cada vuelta y nunca progrese.